CS336 作业 4：数据（Data）

原文：Stanford CS336 作业说明（英文原版）

本文档为英文原版作业说明的非官方中文翻译，仅供学习参考；如与原文有歧义，请以英文原文为准。

目录

1 作业概述 **2**

2 过滤 Common Crawl **2**

2.1 观察数据 3

2.2 HTML 转文本转换 4

2.3 语言识别 5

2.4 个人可识别信息 6

2.5 有害内容 7

2.6 质量规则 8

2.7 质量分类器 9

3 去重 **10**

3.1 精确行去重 10

3.2 MinHash + LSH 文档去重 11

4 为语言建模过滤数据 **13**

1 作业概述

在本作业中，你将通过过滤网络爬取（web crawl）数据来构造语言建模数据，从而获得一些动手经验。

你要实现的内容。

1. 将 Common Crawl 的 HTML 转换为文本。
2. 用多种方法过滤提取出的文本（例如有害内容、个人可识别信息等）。
3. 对训练数据去重。

你要运行的内容。

1. 在不同数据集上训练语言模型，以更好地理解特定的数据处理决策对性能的影响。

代码的结构。所有作业代码以及本说明文档都可以在 GitHub 上获取：github.com/stanford-cs336/assignment4-data

请 `git clone` 该仓库。如果有任何更新，我们会通知你，你可以通过 `git pull` 获取最新版本。

1. `cs336_basics/*` ：该文件夹包含训练我们在作业 1 中构建的模型的代码。模型经过了轻微优化，把作业 1 中一些手工打造的组件替换成了 PyTorch 原生对应的实现（例如，使用 PyTorch 内置的交叉熵 kernel）。我们还提供了一个支持多 GPU 分布式数据并行训练的训练脚本。你将使用该脚本在过滤后的数据上训练模型。
2. `cs336_data/*` ：这是你为作业 4 编写代码的地方。我们创建了一个名为 `cs336_data` 的模块，其中包含本作业的起始代码（starter code）和占位符。
3. `tests/*.py` ：这里包含你必须通过的全部测试。这些测试会调用 `tests/adapters.py` 中定义的钩子（hook）。你需要实现这些 adapter，把你的代码与测试连接起来。多写测试和/或修改测试代码对调试有帮助，但你的实现预期要通过原始提供的测试套件。
4. `README.md` ：该文件包含关于预期目录结构的更多细节，以及一些配置环境的基本说明。

如何提交。你需要把以下文件提交到 Gradescope：

- `writeup.pdf` ：回答所有书面问题。请对你的回答进行排版。
- `code.zip` ：包含你编写的全部代码。

2 过滤 Common Crawl

尽管大语言模型主要在互联网数据上训练，但大多数研究者并不会专门为自己的模型构建网络爬虫来获取训练数据，而是使用公开可用的爬取结果。最流行的公开网络爬取数据来自 Common

Crawl——一个提供免费网页语料的非营利组织，它提供了“横跨 17 年的超过 2500 亿个网页”。[1]

然而，正如我们将看到的，要把 Common Crawl（CC）的转储数据（dump）变成可用于语言模型训练的数据，需要大量的工作。例如，网页的原始数据是 HTML 格式，而我们需要从中提取文本。此外，许多页面可能质量低下、完全或近似重复、包含有害内容或敏感信息，我们可能希望把这些页面过滤掉，或者从数据集中移除其内容中不理想的部分。在本作业中，我们将搭建一个执行其中若干步骤的流水线，把原始互联网数据变成可用的语言模型训练集。

2.1 观察数据

在动手实现任何东西之前，先观察原始数据、对它有个直观感受总是有益的。CC 数据以三种格式提供：

WARC（“Web ARChive format”，Web 归档格式）文件包含原始的 CC 数据，其中包括页面 ID 和 URL、元数据以及 HTTP 请求细节（例如请求的日期与时间、服务器 IP 地址），当然还有页面的原始内容（例如 HTML）。

WAT（“Web Archive Transformation”）文件包含更高层次的元数据，它们从 WARC 文件中提取，并以 JSON 对象的形式存储。例如，对 HTML 页面而言，其中包括该页面中的链接列表和页面标题。

WET（“Web Extracted Text”）文件包含从原始 HTML 页面中提取出的纯文本。

在接下来的问题中，我们将查看一个 WARC 文件及其对应的 WET 文件。这些文件属于 2026 年的一次 Common Crawl 爬取，下载方式如下：

```
# 下载一个示例 WARC 文件。
$ wget https://data.commoncrawl.org/crawl-data/CC-MAIN-2026-12/segments/1741613651012.6/warc/CC-
    MAIN-20260310152245-20260310182245-00000.warc.gz
# 下载其对应的 WET 文件。
$ wget https://data.commoncrawl.org/crawl-data/CC-MAIN-2026-12/segments/1741613651012.6/wet/CC-
    MAIN-20260310152245-20260310182245-00000.warc.wet.gz
```

这些文件可以在以下位置获取：

- `/shared-data/CC/example.warc.gz`
- `/shared-data/CC/example.warc.wet.gz`

警告：这些文件包含完全未经过滤的互联网页面，其中可能包含大量潜在有害内容。如果你看到不愿意阅读的文档，尽管继续向下翻页，跳到下一个。

问题（`look_at_cc`）：观察 Common Crawl（4 分）

(a) 使用上面给出的路径下的 WARC 文件副本。让我们看看这个文件中的第一个页面。这是一个 gzip 压缩文件，你可以用如下命令浏览其内容：

[1] https://commoncrawl.org/

```
$ zcat /shared-data/CC/example.warc.gz | less
```

`less` 允许你用键盘方向键、Page Up、Page Down 来浏览文件。要退出，请按“q”。查看最开头的那个网页。它的 URL 是什么？它现在还能访问吗？仅通过查看原始 HTML，你能看出这个页面大概是关于什么的吗？

交付要求（Deliverable）：2-3 句话的回答。

(b) 现在让我们看看对应的 WET 文件：

```
$ zcat /shared-data/CC/example.warc.wet.gz | less
```

注意 WET 文件包含不属于提取后文本内容的 HTTP 头部（例如 `Content-Length` ）。如果查看第一个样例，你会看到其中包含从你刚才看到的原始 HTML 中提取出的文本。

请注意，许多提取出的文本让人联想到 HTML 结构，而并非页面真正的主内容。你认为你看到的文本中，是否有本应被提取器过滤掉的部分？从训练数据的角度思考这段文本的质量：在这样的文本上训练模型可能出现什么问题？反过来，模型又可能从该页面中提取到什么有用的信息？

交付要求：3-4 句话的回答。

(c) 什么样的样本是好的训练样本高度依赖于具体场景。请描述一个应用领域，在这个领域中把该样例放进训练数据可能有用；再描述一个它可能没用的领域。

交付要求：1-2 句话的回答。

(d) 让我们多看一些例子，以便更好地了解 Common Crawl 里都有什么。再浏览 25 条 WET 记录。对每条记录，非常简短地评论文档的语言（如果你能识别的话）、域名、页面类型等。到你看到一个你认为“高质量”的网页为止，一共需要看多少个例子？

交付要求：对 25 个文档的简要标注，包括文档的语言、域名、页面类型，以及关于该文档的其他杂项备注；以及看到高质量样例之前所需的例子数量。

2.2 HTML 转文本转换

正如你从上面查看 WARC 和 WET 文件的过程中可能已经意识到的，从 HTML 中提取文本颇具挑战。通常，任何提取过程都会在 HTML 中寻找可见内容（例如 `<p>` 标签，它们本应包含文本块）。但即便如此，提取出的内容仍然可能远多于我们在浏览器中打开页面时所感知的页面主内容。例如，打开 StackOverflow 时，主内容是问题和答案，但从技术上讲，菜单选项、指向其他 StackExchange 站点无关页面的链接、页脚、注册或登录链接——这些都是可见文本，要可靠地把它们与页面主内容区分开很有挑战。

许多工具都实现了文本提取流水线。在本作业中，我们将使用 Resiliparse 库来进行文本提取。[2] Resiliparse 还能帮助解决一个更基础的问题：检测承载原始内容的字节所使用的文本编码。虽然

[2] https://resiliparse.chatnoir.eu/en/stable/index.html

Web 上大多数页面都采用 UTF-8 编码（据 Wikipedia 统计为 98.2%），我们的文本提取流水线对其他编码也应当具有健壮性。

注意：我们建议使用 FastWARC 库来遍历每个 WARC 文件中的记录。具体来说，以下类可能会有帮助：

```
from fastwarc.warc import ArchiveIterator, WarcRecordType
```

问题（`extract_text`）：HTML 转文本转换（3 分）

(a) 编写一个函数，从包含原始 HTML 的字节串（byte string）中提取文本。请使用 `resiliparse.extract.html2text.extract_plain_text` 来执行提取。该函数需要传入字符串，所以你需要先把字节串解码成 Unicode 字符串。注意输入字节串可能并非以 UTF-8 编码，因此你的函数应当能够在 UTF-8 解码失败的情况下检测编码。Resiliparse 还提供了 `resiliparse.parse.encoding.detect_encoding()`，可能会有用。

交付要求：一个函数，接受包含 HTML 的字节串，返回包含提取出的文本的字符串。请实现 adapter `[run_extract_text_from_html_bytes]`，并确保它通过 `uv run pytest -k test_extract_text_from_html_bytes`。

(b) 在单个 WARC 文件上运行你的文本提取函数。将其输出与对应 WET 文件中的提取文本进行比较。你注意到哪些差异和/或相似之处？哪种提取看起来更好？

交付要求：2-3 句话的回答，对你自己的函数提取出的文本与 WET 文件中的提取文本进行比较和对照。

2.3 语言识别

Web 上包含用数千种语言写成的页面。但在大多数算力预算下，训练一个能在规模上有效利用如此多样化数据的多语言模型很有挑战。因此，许多源自 Common Crawl 的语言模型训练集只包含有限几种语言的数据。

fastText（https://fasttext.cc）是一个用于此目的的实用库，它提供高效的文本分类器。该库既提供了在你自己的数据上训练分类器的基础设施，也提供了一批预训练模型，其中包括语言识别模型。你可以从 https://fasttext.cc/docs/en/language-identification.html 下载 fastText 语言识别模型 `lid.176.bin`；它也可以在 `/shared-data/classifiers/lid.176.bin` 获取。

通常，语言过滤器会利用分类器给出的分数来决定是否保留某个页面。请使用 fastText 语言识别分类器实现一个语言识别过滤器，它应当为其预测的置信度给出一个非负分数。

问题（`language_identification`）：语言识别（6 分）

(a) 编写一个函数，接受一个 Unicode 字符串，识别该字符串中出现的主要语言。你的函数应当返回一个二元组，其中包含语言的标识符和一个 0 到 1 之间的分数，表示对该预测的置信度。

交付要求：一个执行语言识别的函数，给出其首要的语言预测及分数。请实现 adapter `[run_identify_language]`，并确保它通过 `uv run pytest -k test_identify_language` 中的两个

测试。注意这些测试假定英语（“en”）和中文（“zh”）使用特定的字符串标识符，因此如有必要，你的测试 adapter 应当执行相应的重映射。

(b) 语言模型在推理时的行为很大程度上取决于它们训练时所用的数据。因此，数据过滤流水线中的问题可能在下游引发问题。你认为语言识别流程中的问题可能引发哪些问题？在风险更高的场景中（例如部署面向用户的产品时），你会如何缓解这些问题？

交付要求：2-5 句话的回答。

(c) 在从 WARC 文件中提取的文本上（通过你之前实现的文本提取函数）运行你的语言识别系统。手动识别 20 个随机样本的语言，并将你的标注与分类器的预测进行比较。报告所有分类器错误。文档中英语文档占多大比例？根据你的观察，过滤时使用什么分类器置信度阈值比较合适？

交付要求：2-5 句话的回答。

2.4 个人可识别信息

Web 上包含大量可用于联系或识别个人的信息，例如电子邮件地址、电话号码或 IP 地址。我们可能不希望面向用户的语言模型输出关于真实人物的这类信息，一个常见步骤是在训练数据集中把这些信息遮蔽（mask）掉。

你现在要实现三个遮蔽过程，分别针对 (a) 电子邮件地址、(b) 电话号码和 (c) IP 地址。

问题（`mask_pii`）：个人可识别信息（3 分）

(a) 编写一个函数来遮蔽电子邮件地址。你的函数以字符串为输入，把所有电子邮件地址的实例替换为字符串 `"|||EMAIL_ADDRESS|||"`。要检测电子邮件地址，你可以查找能可靠完成此事的正则表达式。

交付要求：一个函数，把给定字符串中的所有电子邮件地址替换为字符串 `"|||EMAIL_ADDRESS|||"`，返回一个二元组，同时包含新字符串和被遮蔽实例的数量。请实现 adapter `[run_mask_emails]`，并确保它通过 `uv run pytest -k test_mask_emails` 中的所有测试。

(b) 编写一个函数来遮蔽电话号码。你的函数以字符串为输入，把所有电话号码的实例替换为字符串 `"|||PHONE_NUMBER|||"`。要可靠地做到这一点极其困难，因为电话号码的书写格式极其多样，但你至少应当尝试覆盖美国最常用的电话号码格式，并对轻微的语法变体保持健壮。

交付要求：一个函数，把给定字符串中的电话号码替换为字符串 `"|||PHONE_NUMBER|||"`，返回一个二元组，同时包含新字符串和被遮蔽实例的数量。请实现 adapter `[run_mask_phone_numbers]`，并确保它通过 `uv run pytest -k test_mask_phones`。

(c) 编写一个函数来遮蔽 IP 地址。对于此问题，只关注 IPv4 地址（由点分隔的 4 个不超过 255 的数字）就足够了。你的函数以字符串为输入，把所有 IP 地址的实例替换为字符串 `"|||IP_ADDRESS|||"`。

交付要求：一个函数，把给定字符串中的 IPv4 地址替换为字符串 `"|||IP_ADDRESS|||"` ，返回一个二元组，同时包含新字符串和被遮蔽实例的数量。请实现 adapter `[run_mask_ips]` ，并确保它通过 `uv run pytest -k test_mask_ips` 。

(d) 当这些过滤器被简单地（naïvely）应用于训练集时，你认为可能在下游的语言模型中引发什么问题？你会如何缓解这些问题？

交付要求：2-5 句话的回答。

(e) 在从 WARC 文件中提取的文本上（通过你之前实现的文本提取函数）运行你的 PII 遮蔽函数。查看 20 个执行了替换的随机样本；给出一些假阳性（false positive）和假阴性（false negative）的例子。

交付要求：2-5 句话的回答。

2.5 有害内容

未经过滤的 Web 转储数据包含大量我们不希望语言模型在推理时复述出来的文本。其中一些训练样本甚至可能出人意料地来自大体无害的网站（例如 Wikipedia）——比如，用户在某些页面上留下的评论可能相当恶毒（toxic）。尽管要围绕“什么是有害的”划出一条清晰界线本质上是不可能的，许多数据过滤流水线仍然会对主要包含有害内容的页面进行一定过滤。

识别此类内容的方法有很多，包括统计来自禁用词表的词语，或基于人类标注员给出的标签构建简单分类器。在作业的这一部分，我们将专注于识别两大类有害内容：“不适合工作场所”（Not Safe for Work，NSFW）内容（包括色情、亵渎语言或其他潜在令人不适的内容），以及毒性言论（“粗鲁、不尊重或不讲理的、可能让某人退出讨论的言论”）。[3]我们将使用 Dolma 项目 [1] 提供的 fasttext 预训练模型来判断一段输入文本是否属于上述类别之一。这些分类器在 Jigsaw Toxic Comments 数据集上训练，该数据集包含按多种标签归类的 Wikipedia 评论。[4]

NSFW 分类器可从 `dolma-artifacts.org/.../jigsaw_fasttext_bigrams_nsfw_final.bin` 下载。

仇恨言论分类器位于 `dolma-artifacts.org/.../jigsaw_fasttext_bigrams_hatespeech_final.bin` 。

我们还把这两个分类器放在了共享数据目录中：

- `/shared-data/classifiers/dolma_fasttext_hatespeech_jigsaw_model.bin` 是一个用于仇恨与毒性言论的预训练分类器；
- `/shared-data/classifiers/dolma_fasttext_nsfw_jigsaw_model.bin` 是一个用于 NSFW 内容的预训练分类器。

请使用这些模型实现一个函数：接受包含页面内容的 Unicode 字符串，返回一个标签（例如“toxic”（有毒）、“non-toxic”（无毒））以及一个置信度分数。

[3] https://current.withgoogle.com/the-current/toxicity/

[4] https://paperswithcode.com/dataset/toxic-comment-classification-challenge

问题（`harmful_content`）：有害内容（6 分）

(a) 编写一个函数来检测 NSFW 内容。

交付要求：一个函数，标注给定字符串是否包含 NSFW 内容，返回一个二元组，同时包含标签和置信度分数。请实现 adapter `[run_classify_nsfw]` ，并确保它通过 `uv run pytest -k test_classify_nsfw` 。注意这个测试只是取自 Jigsaw 数据集的健全性检查（sanity check），绝不能断言你的分类器是准确的——你需要自行验证。

(b) 编写一个函数来检测毒性言论。

交付要求：一个函数，标注给定字符串是否为毒性言论，返回一个二元组，同时包含标签和置信度分数。请实现 adapter `[run_classify_toxic_speech]` ，并确保它通过 `uv run pytest -k test_classify_toxic_speech` 。同样，这个测试也只是取自 Jigsaw 的健全性检查。

(c) 当这些过滤器被用于构建训练集时，你认为可能在下游的语言模型中引发什么问题？你会如何缓解这些问题？

交付要求：2-5 句话的回答。

(d) 在从 WARC 文件中提取的文本上（通过你之前实现的文本提取函数）运行你的有害内容过滤器。查看 20 个随机样本，并将分类器的预测与你自己的判断进行比较。报告所有分类器错误。文档中有害文档占多大比例？根据你的观察，过滤时使用什么分类器置信度阈值比较合适？

交付要求：2-5 句话的回答。

2.6 质量规则

即使在按语言过滤页面并移除有害内容之后，保留下来的页面中仍有相当大一部分对语言模型训练而言质量低下。同样，虽然“质量”并不容易定义，浏览 Common Crawl 的样例有助于识别低质量内容的例子，例如：

- 带付费墙内容的页面；
- 失效链接的占位页面；
- 登录、注册或联系表单；
- 以非文本内容为主的页面，它们在文本提取过程中会丢失（例如照片、视频）。

Gopher 论文 [2] 描述了一组简单的质量过滤器，用于从网页抓取数据中移除类似的简单低质量文本。这些过滤器由易于理解的简单启发式规则组成，通常能覆盖许多明显不适用的样例。Gopher 质量过滤器包括若干基于文档长度、词长、符号—词比率以及特定英文停用词是否出现的准则。在本作业中，你将实现 Gopher 论文 [2] 中所述过滤器的一个子集。具体而言，你应当移除满足以下条件的文档：

- 词数少于 50 或多于 100,000 的文档；

- 平均词长不在 3 到 10 个字符范围内的文档；
- 超过 30% 的行以省略号（“...”）结尾的文档；
- 至少含有一个字母字符的词占比低于 80% 的文档。

关于 Gopher 论文所使用的全部质量过滤器的完整描述，请参阅其附录 A。

问题（`gopher_quality_filters` ）：Gopher 质量过滤器（3 分）

(a) 按上述说明实现（至少）这一 Gopher 质量过滤器子集。要把文本切分成词，你可能会发现 NLTK 包很有用（具体是 `nltk.word_tokenize` ），但并不强制使用它。

交付要求：一个函数，以字符串为唯一参数，返回一个布尔值，指示该文本是否通过 Gopher 质量过滤器。请实现 adapter `[run_gopher_quality_filter]` 。然后，确保你的过滤器通过 `uv run pytest -k test_gopher` 中的测试。

(b) 在从 WARC 文件中提取的文本上（通过你之前实现的文本提取函数）运行你基于规则的质量过滤器。查看 20 个随机样本，并将过滤器的预测与你自己的判断进行比较。对质量过滤器与你的判断不一致的所有情况进行点评。

交付要求：2-5 句话的回答。

2.7 质量分类器

现在让我们超越 Gopher 规则所刻画的简单语法层面的质量标准。语言模型训练并不是第一个需要按质量对内容排序的领域。特别地，文本质量也是信息检索中的一项根本性挑战。搜索引擎利用的一个经典信号是 Web 上链接的结构：高质量页面往往链接到其他高质量页面 [3]。OpenAI 在构建 WebText（GPT-2 [4] 的训练数据集）时使用了类似的洞见：他们收集被 Reddit 评论链接、且高于最低“karma”门槛的页面。Reddit 的一种替代方案是把 Wikipedia 用作高质量链接的来源，因为 Wikipedia 页面所链接的外部来源往往是受信任的页面 [5]。

虽然使用受控来源通常能带来高质量内容，但按今天的标准，所得数据集的规模偏小（OpenWebText 有 40GB 文本，而 The Pile 比它大 20 倍以上）。一种做法是把上述参考页面用作正样本，把（随机抽取的）Common Crawl 页面用作负样本，训练一个 fastText 分类器。该分类器给出一个质量分数，你可以用它从整个 Common Crawl 中过滤页面。设置质量阈值就是在精确率（precision）与召回率（recall）之间做权衡。

在作业的这一部分，你将构建一个质量分类器。为方便起见，我们已从近期的一个 Wikipedia 转储中提取了参考页面的 URL，并把它们放在共享数据目录的 `/shared-data/wiki/enwiki-20260501-extracted_urls.txt.gz` 。该文件包含截至 2026 年 5 月英文 Wikipedia 页面上发现的外部链接，但我们希望你对这些 URL 进行子采样，以获得用于训练分类器的“高质量”文本正样本。注意这些正样本可能仍包含不理想的内容，因此，应用你已经构建的其他基础组件（例如语言识别、过滤规则等）来进一步提升其质量可能会有用。给定一个 URL 文件，你可以用 `wget` 以 WARC 格式抓取其内容：

```
wget --timeout=5 \
       -i subsampled_positive_urls.txt \
       --warc-file=subsampled_positive_urls.warc \
       -O /dev/null
```

问题（quality_classifier ）：质量分类器（15 分）

(a) 训练一个质量分类器：给定文本，返回一个数值型的质量分数。

交付要求：一个质量分类器，供下一个子问题使用。

(b) 编写一个函数，把页面标注为高质量或低质量，并为该标注提供置信度分数。

交付要求：一个函数，以字符串为唯一参数，返回一个二元组：标签（高质量或不是）和置信度分数。请实现 adapter [run_classify_quality] 。作为健全性检查，通过运行 uv run pytest -k test_classify_quality ，确保它能正确分类我们提供的两个样例。

3 去重

Web 上包含大量重复内容。有些页面彼此完全相同——想想存档页面，或者由标准工具生成的默认页面，例如流行 Web 服务器的 404 页面。但大多数重复发生在更细的粒度上。例如，考虑 Stack Overflow 上的所有问题页面：虽然每个页面都有独特的内容（例如问题、评论、答案本身），但所有页面也都有大量冗余，例如页眉、菜单选项和页脚——当这类页面被渲染时，它们会以完全相同的形式重复出现。在本节的第一部分，我们将处理这种完全重复，稍后再看如何处理近似重复。

3.1 精确行去重

对完全重复进行去重的一种简单方法是：只保留文档中在语料库范围内唯一的行。事实证明，这足以消除很大一部分冗余，例如我们上面提到的页眉和菜单选项。在较简单的情况下，移除在其他地方精确重复出现的行之后，剩下的往往就是每个页面独特的主内容（例如 StackOverflow 上的问题和答案）。

为此，我们可以先遍历一遍语料库，统计每行出现的次数；然后在第二遍中重写每个文档，只保留其中唯一的行。朴素做法中，保存计数器的数据结构所占用的空间可能与存储语料库中所有唯一的行所需的空间一样大。一个简单的内存效率技巧是改用该行的哈希值作为键，让键具有固定大小（而不是取决于行的长度）。你现在将实现这个简单的去重方法。

问题（exact_deduplication ）：精确行去重（3 分）

编写一个函数，接受一组输入文件路径作为输入，对它们执行精确行去重。它应当先统计语料库中每行出现的频率（使用哈希来减少内存占用），然后重写每个文件，只保留其中唯一的行。

交付要求：一个执行精确行去重的函数。你的函数应当接受两个参数：(a) 输入文件路径的列表，以及 (b) 一个输出目录。它应当把每个输入文件以相同的名字重写到输出目录，但通过移除

在输入文件集合中出现多于一次的行来对内容去重。例如，如果输入路径是 `a/1.txt` 和 `a/2.txt`，输出目录是 `b/`，你的函数应当写出文件 `b/1.txt` 和 `b/2.txt`。请实现 adapter `[run_exact_line_deduplication]`，并确保它通过 `uv run pytest -k test_exact_line_deduplication`。

3.2 MinHash + LSH 文档去重

精确去重有助于移除在多个网页间逐字重复的内容，但无法处理文档内容略有差异的情况。例如，考虑软件许可证文档——许可证文档通常由一个模板生成，模板要求填入年份和软件作者的名字。结果，一个采用 MIT 许可证的项目的许可证文件与另一个 MIT 许可项目的许可证文件内容大体相同，但它们并非完全相同的副本。要移除这类重复的、大部分基于模板的内容，我们需要模糊去重（fuzzy deduplication）。为了高效地执行文档级的模糊去重，我们将使用带局部敏感哈希（locality sensitive hashing，LSH）的 minhash。[5]

为了进行模糊去重，我们将使用文档之间相似性的一种特定度量：各文档 n-gram 之间的 Jaccard 相似度。集合 S 与 T 之间的 Jaccard 相似度定义为

$$J(S,T) = \frac{|S \cap T|}{|S \cup T|}. \tag{1}$$

要以朴素的方式执行模糊去重，我们可以把每个文档表示成一个 n-gram 集合，并计算所有文档对之间的 Jaccard 相似度，把超过某个 Jaccard 相似度阈值的文档对标记为重复。然而，这种方法对大规模文档集合（例如 Common Crawl）并不现实。此外，朴素地存储 n-gram 集合所占用的内存将远超文档本身。

MinHash（最小哈希）

为了解决内存问题，我们用签名（signature）来替代 n-gram 集合这一文档表示。具体而言，我们希望构造这样的签名：比较两个文档的签名，就能得到这两个文档各自 n-gram 集合之间 Jaccard 相似度的近似。minhash 签名正满足这些性质。要计算文档 n-gram 集合 $S = \{s_1, \ldots, s_n\}$ 的 minhash 签名，我们需要 k 个互不相同的哈希函数 $h_1, \ldots, h_k$。[6]每个哈希函数把一个 n-gram 映射为一个整数。给定哈希函数 h_i，文档 n-gram 集合 S 的 minhash 为

$$\text{minhash}(h_i, S) = \min\big(h_i(s_1),\ h_i(s_2),\ \ldots,\ h_i(s_n)\big). \tag{2}$$

文档 n-gram 集合 S 的签名是 $\mathbb{R}^k$ 中的一个向量，其中第 i 个元素是 S 在随机哈希函数 h_i 下的 minhash，即

$$\big[\text{minhash}(h_1, S),\ \text{minhash}(h_2, S),\ \ldots,\ \text{minhash}(h_k, S)\big]. \tag{3}$$

事实证明，对两个文档的 n-gram 集合 S_1 与 S_2，集合之间的 Jaccard 相似度可以由两签名中取值相同的列所占的比例来近似（证明见 [6] 第 3 章第 3.3.3 节）。例如，给定文档签名 $[1,2,3,2]$ 和 $[5,2,3,4]$，n-gram 集合之间的相似度近似为 $2/4$，因为这两个签名的第二列和第三列具有相同的 minhash 值。

[5]关于 LSH 和 minhash 的更深入论述，读者可参阅 J. Leskovec、A. Rajaraman 和 J. D. Ullman 的教材 [6] 第 3 章。该章可在线获取：infolab.stanford.edu/ullman/mmds/ch3n.pdf。

[6]这 k 个互不相同的哈希函数可以来自同一函数族，但使用不同的种子。例如，MurmurHash3 是一个哈希函数族，使用一个特定的种子即在该族中实例化出一个特定的哈希函数。

局部敏感哈希（LSH）

尽管 minhash 为我们提供了一种节省内存、并且保持任意文档对期望相似度的文档表示，我们仍然需要比较所有文档对，才能找出相似度最高的那些。LSH 提供了一种把可能具有高相似度的文档高效分桶的方法。要把 LSH 应用于我们的文档签名（现在是 $\mathbb{R}^k$ 中的一个向量），我们将把签名划分为 b 个波段（band），每个波段包含 r 个 minhash，且 $k = br$。例如，如果有 100 个元素的文档签名（由 100 个随机哈希函数生成），我们可以把它拆成 2 个各含 50 个 minhash 的波段，或 4 个各含 25 个 minhash 的波段，或 50 个各含 2 个 minhash 的波段，等等。如果两个文档在某个特定波段上具有相同的哈希值，它们就会被聚到同一个桶中，并被视为候选重复。因此，在签名长度固定的情况下，增加波段数量会提高召回率（recall）、降低精确率（precision）。

举一个上述内容的具体例子：假设文档 D_1 的 minhash 签名为 $[1, 2, 3, 4, 5, 6]$，另一个文档 D_2 的 minhash 签名为 $[1, 2, 3, 5, 1, 2]$。如果我们使用 3 个波段、每个波段 2 个 minhash，那么 D_1 的第一个波段是 $[1, 2]$，第二个波段是 $[3, 4]$，第三个波段是 $[5, 6]$；类似地，D_2 的第一个波段是 $[1, 2]$，第二个波段是 $[3, 5]$，第三个波段是 $[1, 2]$。由于第一个波段中的哈希值匹配（两个文档都是 $[1, 2]$），D_1 和 D_2 会在第一个波段下被聚到一起；而在其他任何波段下它们都不会被聚到一起，因为哈希值不匹配。不过要注意，由于两个文档至少在一个波段下被聚在一起，无论其他波段是否匹配，它们都会被视为候选重复。

找出候选重复之后，我们可以用多种方式处理它们。例如，我们可以计算所有候选重复之间的 Jaccard 相似度，把超过设定阈值的文档对标记为重复。

最后，我们跨桶对重复文档进行聚类。例如，假设文档 A 和 B 在一个桶中匹配，且它们的真实 Jaccard 相似度高于我们的阈值；同时文档 B 和 C 在另一个桶中匹配，且它们的真实 Jaccard 相似度也高于阈值。那么，我们会把文档 A、B、C 视为同一个簇。我们在每个簇中随机保留一个文档，移除其余文档。

问题（`minhash_deduplication`）：MinHash + LSH 文档去重（8 分）

编写一个函数，接受一组输入文件路径作为输入，使用 minhash 和 LSH 执行模糊文档去重。具体而言，你的函数应当为给定路径列表中的每个文档计算 minhash 签名，使用给定数量的波段进行 LSH 以找出候选重复，然后计算候选重复之间真实的 n-gram Jaccard 相似度，并移除超过给定阈值的文档。为了提高召回率（依照 [7]），在计算 minhash 签名和/或比较 Jaccard 相似度之前，请对文本做归一化：转为小写、移除标点、归一化空白字符、移除重音符号，并进行 NFD Unicode 归一化。

交付要求：一个执行模糊文档去重的函数。你的函数应当至少接受以下参数：(a) 输入文件路径的列表，(b) 用于计算 minhash 签名的哈希函数数量，(c) 用于 LSH 的波段数量，(d) 用于计算 minhash 签名的 n-gram 长度（以词为单位），以及 (e) 一个输出目录。你可以假定用于计算 minhash 签名的哈希函数数量能被用于 LSH 的波段数量整除。

你的函数应当把每个输入文件以相同的名字重写到输出目录，但只写出满足以下条件之一的文档：(a) 不是候选重复，和/或 (b) 从聚类得到的桶中被随机选中保留的文档。例如，如果输入路径是 `a/1.txt` 和 `a/2.txt`，输出目录是 `b/`，你的函数应当写出文件 `b/1.txt` 和 `b/2.txt`。

请实现 adapter `[run_minhash_deduplication]` ，并确保它通过 `uv run pytest -k test_minhash_deduplication` 。

4 为语言建模过滤数据

既然我们已经实现了多种用于过滤网络爬取数据的基础组件，现在就把它们用起来，生成一些语言建模训练数据吧。

在作业的这一部分，你的目标是过滤一组 CC WET 文件，以生成语言建模训练数据。你将从已经过英语过滤的 WET 文件开始：我们保留了根据 fastText `lid.176.bin` 语言识别模型英语概率不低于 70% 的文档。我们已把这些英语过滤后的 WET 文件放在 `/shared-data/english-wet-data` 下，供你作为起点。

具体而言，你的目标是过滤 CC 转储数据以创建语言建模数据，使得据此训练出的 Transformer 语言模型在 Paloma 基准 [8] 的 C4 100 domains 子集上的验证困惑度（perplexity）最小。你不应修改模型架构或训练流程，因为这里的目标是构造出最好的数据。该数据集包含 C4 语言建模数据集 [9] 中最常见 100 个域名的样本。我们已把该数据的一份副本放在 `/shared-data/tokenized_paloma_c4_100_domains_validation.bin` （已用 GPT-2 分词器分词）——你可能愿意浏览一下这些数据，感受一下它的样子。你可以这样加载它：

```
import numpy as np
data = np.fromfile(
    "/shared-data/tokenized_paloma_c4_100_domains_validation.bin",
    dtype=np.uint16
)

from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("gpt2")
print(tokenizer.decode(data[0:2000]))
```

我们提醒：允许你在构建过滤器或分类器来处理 CC WET 文件时使用 Paloma 验证数据，但不允许把任何验证数据逐字复制到你的训练数据中。语言模型绝不应看到来自验证集的任何数据。

即便只是这一批 WET 文件也是相当可观的数据量。为了高效处理这些数据，我们建议尽可能地使用多进程（multi-processing）。具体来说，你可能会发现 Python 的 `concurrent.futures` 或 `multiprocessing` API 很有帮助。下面我们给出一个使用 `concurrent.futures` 把函数并行化到多个进程的最小示例：

```
import concurrent.futures
import os

from tqdm import tqdm

def process_single_wet_file(input_path: str, output_path: str):
    # TODO: 读取输入路径，处理输入，并把输出写入 output_path
    return output_path
```

```
# 设置 executor
num_cpus = len(os.sched_getaffinity(0))
executor = concurrent.futures.ProcessPoolExecutor(max_workers=num_cpus)
wet_filepaths = ["a.warc.wet.gz", "b.warc.wet.gz", "c.warc.wet.gz"]
output_directory_path = "/path/to/output_directory/"

futures = []
for wet_filepath in wet_filepaths:
    # 对每个 warc.wet.gz 文件路径，向 executor 提交一个任务并拿回一个 future
    wet_filename = str(pathlib.Path(wet_filepath).name)
    future = executor.submit(
        process_single_wet_file,
        wet_filepath,
        os.path.join(output_directory_path, wet_filename)
    )
    # 保存 futures
    futures.append(future)

# 在 future 完成时对其进行遍历，用进度条跟踪进度
for future in tqdm(
    concurrent.futures.as_completed(futures),
    total=len(wet_filepaths),
):
    output_file = future.result()
    print(f"Output file written: {output_file}")
```

对较大的文本块请使用本地并行机制，例如 `concurrent.futures` 或 `multiprocessing` 。要在 modal 上跨 worker 并行，请对输入调用 `.map(...)` 。

我们还建议使用 FastWARC 库来遍历每个 WET 文件中的记录，并使用 `tldextract` 库从 URL 中提取域名以便过滤。具体来说，这些类可能会有帮助：

```
from fastwarc.warc import ArchiveIterator, WarcRecordType
from tldextract import TLDExtract
```

问题（`filter_data`）：为语言建模过滤数据（6 分）

(a) 编写一个脚本，从一组 Common Crawl WET 文件（位于 `/shared-data/english-wet-data` 下）中过滤出语言建模数据。你可以自由运用我们在作业前面部分实现的任何基础组件，也可以自由探索其他用于生成数据的过滤器和方法（例如基于 n-gram 语言模型困惑度的过滤）。你的目标是产出一批数据，在其上训练后能使 Paloma 基准 C4 100 domains 子集上的困惑度最小。

我们再次提醒：允许你在构建过滤器或分类器来处理 CC WET 文件时使用 Paloma 验证数据，但不允许把任何验证数据逐字复制到你的训练数据中。

你的脚本应当报告你所使用的每个过滤器保留下来的样本数量，这样你就能了解各过滤器对最终输出数据的贡献。

交付要求：一个（或一串）脚本，并行地过滤所提供的 CC WET 文件以生成语言建模数据；以及一份书面分解，说明每个过滤步骤移除了多大比例的被丢弃样本。

(b) 过滤所提供的 WET 文件（原始共 2,500 个 WET 文件）需要多长时间？过滤整个 Common Crawl 转储数据需要多长时间？

交付要求：数据过滤流水线的运行时间。

现在我们已经生成了一些语言建模数据，让我们看一看它，以便更好地理解其内容。

问题（`inspect_filtered_data`）：检查过滤后的数据（4 分）

(a) 从你过滤后的数据集中随机抽取五个样本。点评它们的质量，以及它们是否适合用于语言建模，特别是考虑到我们的目标是在 C4 100 domains 基准上最小化困惑度。

交付要求：最终过滤数据中的五个随机样本。由于文档可能很长，只展示数据中相关的摘录即可。对每个样本，用 1-2 句话描述该样本，以及它是否值得用于语言建模。

(b) 抽取五个被你的过滤脚本移除和/或修改的 CC WET 文档。是你的过滤流程中的哪个环节移除或修改了这些文档？你认为对它们的移除和/或修改是否合理？

交付要求：原始 WET 文件中的五个随机被丢弃样本。由于文档可能很长，只展示数据中相关的摘录即可。对每个样本，用 1-2 句话描述该样本，以及对它的移除是否合理。

(c) 如果你上面的分析促使你对数据流水线做进一步的改动，欢迎在训练模型之前进行这些改动。报告你尝试过的任何数据改动和/或迭代。

交付要求：对你尝试过的数据改动和/或迭代的描述。

在我们的数据上训练语言模型之前，我们需要先对它进行分词（tokenize）。请通过 `transformers` 使用 GPT-2 分词器，把你过滤后的数据编码成整数 ID 序列，用于训练语言模型。别忘了在每个文档后面加上 GPT-2 的序列结束 token `<|endoftext|>` 。下面是一些起始代码：

```
import multiprocessing

import numpy as np
from tqdm import tqdm

from transformers import AutoTokenizer

input_path = "path/to/your/filtered/data"
output_path = "path/to/your/tokenized/data"

tokenizer = AutoTokenizer.from_pretrained("gpt2")

def tokenize_line_and_add_eos(line):
    return tokenizer.encode(line) + [tokenizer.eos_token_id]

with open(input_path) as f:
```

```
    lines = f.readlines()

pool = multiprocessing.Pool(multiprocessing.cpu_count())
chunksize = 100
results = []
for result in tqdm(
    pool.imap(tokenize_line_and_add_eos, lines, chunksize=chunksize),
    total=len(lines),
    desc="Tokenizing lines"
):
    results.append(result)

pool.close()
pool.join()

# 将 id 列表展平并转换为 numpy 数组
all_ids = [token_id for sublist in results for token_id in sublist]
print(f"Tokenized and encoded {input_path} into {len(all_ids)} tokens")
ids_array = np.array(all_ids, dtype=np.uint16)
ids_array.tofile(output_path)
```

问题（tokenize_data）：数据分词（2 分）

编写一个脚本，对你过滤后的数据进行分词并序列化。请确保按照上面的示例代码进行序列化，即使用 `ids_array.tofile(output_path)`，其中 `ids_array` 是一个 `np.uint16` 类型的整数 ID numpy 数组。这能确保与提供的训练脚本兼容。

你过滤后的数据集中有多少个 token？

交付要求：一个对你过滤后的数据进行分词和序列化的脚本，以及你产出的数据集中的 token 数量。

现在我们已经对数据集完成了分词，可以在其上训练模型了。我们将在生成的数据上训练一个 GPT-2 small 形状（small-shaped）、约 430M 参数的模型，并周期性地在 C4 100 domains 数据集上测量验证性能。该训练运行使用 8 块 B200 GPU、数据并行、每设备批量大小 128、16,384 步优化。在上下文长度 512 下，这相当于训练期间采样了约 $2^{33} = 8.6\,\mathrm{B}$ 个训练 token。我们采用此配置的训练运行在大约 2 小时内完成。

使用 `scripts/train.py` 中提供的 `train.py` 脚本启动训练。[7]你可以在 `cs336_basics/train_config.py` 中查看将要使用的超参数。通过 `--train-bin` 传入你的分词后训练数据的路径：

```
uv run modal run scripts/train.py --train-bin /root/data/your_data.bin
```

再次强调，本作业的目标是优化数据以最小化验证损失，而不是通过修改模型和/或优化过程来降低损失，因此请不要修改训练流程或训练脚本。

然后，你可以用以下命令从保存的模型中采样生成：

[7] https://github.com/stanford-cs336/assignment4-data/blob/master/scripts/train.py

```
uv run modal run scripts/generate_with_gpt2_tok.py --model-path /root/data/output/your_data
```

问题（train_model ）：训练模型（8 分）

在你的分词后的数据集上训练一个语言模型（GPT-2 small 形状，约 430M 参数）。周期性地测量 C4 100 domains 上的验证损失。你的模型达到的最佳验证损失是多少？

交付要求：记录到的最佳验证损失、对应的学习曲线，以及对你所做工作的描述。

参考文献

- [1] L. Soldaini et al., "Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research," arXiv preprint arXiv:2402.00159, 2024.
- [2] J. W. Rae et al., "Scaling language models: Methods, analysis & insights from training gopher," arXiv preprint arXiv:2112.11446, 2021.
- [3] L. Page, S. Brin, R. Motwani, and T. Winograd, "The pagerank citation ranking: Bring order to the web," in *Proc. of the 7th International World Wide Web Conf*, 1998.
- [4] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
- [5] H. Touvron et al., "LLaMA: Open and Efficient Foundation Language Models." 2023.
- [6] J. Leskovec, A. Rajaraman, and J. D. Ullman, *Mining of Massive Datasets*, 2nd ed. USA: Cambridge University Press, 2014.
- [7] G. Penedo et al., "The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only." 2023.
- [8] I. Magnusson et al., "Paloma: A Benchmark for Evaluating Language Model Fit." 2023.
- [9] C. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020, [Online]. Available: http://jmlr.org/papers/v21/20-074.html